第11章 持有对象

如果一个程序只包含固定数量的且其生命期都是已知的对象，那么这是一个非常简单的程序。

通常，程序总是根据运行时才知道的某些条件去创建新对象。在此之前，不会知道所需对象的数量，甚至不知道确切的类型。为解决这个普遍的编程问题，需要在任意时刻和任意位置创建任意数量的对象。所以，就不能依靠创建命名的引用来持有每一个对象：

```
MyType aReference;
```

因为你不知道实际上会需要多少这样的引用。

大多数语言都提供某种方法来解决这个基本问题。Java 有多种方式保存对象（应该说是对象的引用）。例如前面曾经学习过的数组，它是编译器支持的类型。数组是保存一组对象的最有效的方式，如果你想保存一组基本类型数据，也推荐使用这种方式。但是数组具有固定的尺寸，而在更一般的情况中，你在写程序时并不知道将需要多少个对象，或者是否需要更复杂的方式来存储对象，因此数组尺寸固定这一限制显得过于受限了。

Java 实用类库还提供了一套相当完整的容器类来解决这个问题，其中基本的类型是**List**、**Set**、**Queue**和**Map**。这些对象类型也称为集合类，但由于Java 的类库中使用了**Collection**这个名字来指代该类库的一个特殊子集，所以我使用了范围更广的术语“容器”称呼它们。容器提供了完善的方法来保存对象，你可以使用这些工具来解决数量惊人的问题。

389

容器还有其他一些特性。例如，**Set**对于每个值都只保存一个对象，**Map**是允许你将某些对象与其他一些对象关联起来的关联数组，Java容器类都可以自动地调整自己的尺寸。因此，与数组不同，在编程时，你可以将任意数量的对象放置到容器中，并且不需要担心容器应该设置为多大。

即使在Java中没有直接的关键字支持[⊖]，容器类仍旧是可以显著增强你的编程能力的基本工具。在本章中，你将了解有关Java容器类库的基本知识，以及对典型用法的重点介绍。我们聚焦于你在日复一日的编程工作中将会用到的那些容器。稍后，在第17章，还将学习到剩余的那些容器，以及有关它们的功能和如何使用它们的更多细节。

11.1 泛型和类型安全的容器

使用Java SE5之前的容器的一个主要问题就是编译器允许你向容器中插入不正确的类型。例如，考虑一个**Apple**对象的容器，我们使用最基本最可靠的容器**ArrayList**。现在，你可以把**ArrayList**当作“可以自动扩充自身尺寸的数组”来看待。使用**ArrayList**相当简单：创建一个实例，用**add()**插入对象；然后用**get()**访问这些对象，此时需要使用索引，就像数组一样，但是不需要方括号[⊖]。**ArrayList**还有一个**size()**方法，使你可以知道已经有多少元素添加了进来，从而不会不小心因索引越界而引发错误（通过抛出运行时异常，异常将在第12章介绍）。

在本例中，**Apple**和**Orange**都放置在了容器中，然后将它们取出。正常情况下，Java编译器

⊖ 许多语言，例如Perl、Python和Ruby，都有容器的本地支持。

⊖ 这里是操作符重载的用武之地，C++和C#的容器类都使用操作符重载生成了更简洁的语法。

会报告警告信息，因为这个示例没有使用泛型。在这里，我们使用Java SE5所特有的注解来抑制了警告信息。注解以“@”符号开头，可以接受参数，这里的@SuppressWarnings注解及其参数表示只有有关“不受检查的异常”的警告信息应该被抑制：

390

```
//: holding/ApplesAndOrangesWithoutGenerics.java
// Simple container example (produces compiler warnings).
// {ThrowsException}
import java.util.*;

class Apple {
  private static long counter;
  private final long id = counter++;
  public long id() { return id; }
}

class Orange {}

public class ApplesAndOrangesWithoutGenerics {
  @SuppressWarnings("unchecked")
  public static void main(String[] args) {
    ArrayList apples = new ArrayList();
    for(int i = 0; i < 3; i++)
      apples.add(new Apple());
    // Not prevented from adding an Orange to apples:
    apples.add(new Orange());
    for(int i = 0; i < apples.size(); i++)
      ((Apple)apples.get(i)).id();
      // Orange is detected only at run time
  }
} /* (Execute to see output) *///:~
```

在第20章中将会更多地学习有关Java SE5的注解。

Apple和**Orange**类是有区别的，它们除了都是**Object**之外没有任何共性（记住，如果一个类没有显式地声明继承自哪个类，那么它自动地继承自**Object**）。因为**ArrayList**保存的是**Object**，因此你不仅可以通过**ArrayList**的**add()**方法将**Apple**对象放进这个容器，还可以添加**Orange**对象，而且无论在编译期还是运行时都不会有问题。当你在使用**ArrayList**的**get()**方法来取出你认为是**Apple**的对象时，你得到的只是**Object**引用，必须将其转型为**Apple**，因此，需要将整个表达式括起来，在调用**Apple**的**id()**方法之前，强制执行转型。否则，你就会得到语法错误。在运行时，当你试图将**Orange**对象转型为**Apple**时，你就会以前面提及的异常的形式得到一个错误。

391

在第15章中，你将会了解到，使用Java泛型来创建类会非常复杂。但是，应用预定义的泛型通常会很简单。例如，要想定义用来保存**Apple**对象的**ArrayList**，你可以声明**ArrayList<Apple>**，而不仅仅只是**ArrayList**，其中尖括号括起来的是*类型参数*（可以有多个），它指定了这个容器实例可以保存的类型。通过使用泛型，就可以在*编译期*防止将错误类型的对象放置到容器中[⊖]。下面还是这个示例，但是使用了泛型：

```
//: holding/ApplesAndOrangesWithGenerics.java
import java.util.*;

public class ApplesAndOrangesWithGenerics {
  public static void main(String[] args) {
    ArrayList<Apple> apples = new ArrayList<Apple>();
    for(int i = 0; i < 3; i++)
      apples.add(new Apple());
    // Compile-time error:
```

⊖ 在第15章的末尾，你会发现有关这个问题是否很严重的讨论。但是，第15章还将展示Java泛型远不止类型安全的容器这么简单。

```
    // apples.add(new Orange());
    for(int i = 0; i < apples.size(); i++)
      System.out.println(apples.get(i).id());
    // Using foreach:
    for(Apple c : apples)
      System.out.println(c.id());
  }
} /* Output:
0
1
2
0
1
2
*///:~
```

392

现在，编译器可以阻止你将**Orange**放置到**apples**中，因此它变成了一个编译期错误，而不再是运行时错误。

你还应该注意到，在将元素从**List**中取出时，类型转换也不再是必需的了。因为**List**知道它保存的是什么类型，因此它会在调用**get()**时替你执行转型。这样，通过使用泛型，你不仅知道编译器将会检查你放置到容器中的对象类型，而且在使用容器中的对象时，可以使用更加清晰的语法。

这个实例还表明，如果不需要使用每个元素的索引，你可以使用foreach语法来选择**List**中的每个元素。

当你指定了某个类型作为泛型参数时，你并不仅限于只能将该确切类型的对象放置到容器中。向上转型也可以像作用于其他类型一样作用于泛型：

```
//: holding/GenericsAndUpcasting.java
import java.util.*;

class GrannySmith extends Apple {}
class Gala extends Apple {}
class Fuji extends Apple {}
class Braeburn extends Apple {}

public class GenericsAndUpcasting {
  public static void main(String[] args) {
    ArrayList<Apple> apples = new ArrayList<Apple>();
    apples.add(new GrannySmith());
    apples.add(new Gala());
    apples.add(new Fuji());
    apples.add(new Braeburn());
    for(Apple c : apples)
      System.out.println(c);
  }
} /* Output: (Sample)
GrannySmith@7d772e
Gala@11b86e7
Fuji@35ce36
Braeburn@757aef
*///:~
```

393

因此，你可以将**Apple**的子类型添加到被指定为保存**Apple**对象的容器中。

程序的输出是从**Object**默认的**toString()**方法产生的，该方法将打印类名，后面跟随该对象的散列码的无符号十六进制表示（这个散列码是通过**hashCode()**方法产生的）。你将在第17章中了解有关散列码的内容。

练习1：(2) 创建一个新类**Gerbil**（沙鼠），包含**int gerbilNumber**，在构造器中初始化它（类似于本章的**Mouse**示例）。添加一个方法**hop()**，用以打印沙鼠的号码以及它正在跳跃的信息。

创建一个**ArrayList**，并向其中添加一串**Gerbil**对象。使用**get()**遍历**List**，并且对每个**Gerbil**调用**hop()**。

11.2 基本概念

Java 容器类类库的用途是“保存对象”，并将其划分为两个不同的概念：

1) **Collection**。一个独立元素的序列，这些元素都服从一条或多条规则。**List** 必须按照插入的顺序保存元素，而Set不能有重复元素。**Queue**按照排队规则来确定对象产生的顺序(通常与它们被插入的顺序相同)。

2) **Map**。一组成对的“键值对”对象，允许你使用键来查找值。**ArrayList**允许你使用数字来查找值，因此在某种意义上讲，它将数字与对象关联在了一起。映射表允许我们使用另一个对象来查找某个对象，它也被称为“关联数组”，因为它将某些对象与另外一些对象关联在了一起；或者被称为“字典”，因为你可以使用键对象来查找值对象，就像在字典中使用单词来定义一样。**Map**是强大的编程工具。

尽管并非总是这样，但是在理想情况下，你编写的大部分代码都是在与这些接口打交道，并且你唯一需要指定所使用的精确类型的地方就是在创建的时候。因此，你可以像下面这样创建一个**List**：

```
List<Apple> apples = new ArrayList<Apple>();
```

注意，**ArrayList**已经被向上转型为**List**，这与前一个示例中的处理方式正好相反。使用接口的目的在于如果你决定去修改你的实现，你所需的只是在创建出修改它，就像下面这样： 394

```
List<Apple> apples = new LinkedList<Apple>();
```

因此，你应该创建一个具体类的对象，将其转型为对应的接口，然后在其余的代码中都使用这个接口。

这种方式并非总能奏效，因为某些类具有额外的功能，例如，**LinkedList**具有在**List**接口中未包含的额外方法，而**TreeMap**也具有在**Map**接口中未包含的方法。如果你需要使用这些方法，就不能将它们向上转型为更通用的接口。

Collection接口概括了序列的概念——一种存放一组对象的方式。下面这个简单的示例用**Integer**对象填充了一个**Collection**（这里用**ArrayList**表示），然后打印所产生的容器中的所有元素：

```
//: holding/SimpleCollection.java
import java.util.*;

public class SimpleCollection {
  public static void main(String[] args) {
    Collection<Integer> c = new ArrayList<Integer>();
    for(int i = 0; i < 10; i++)
      c.add(i); // Autoboxing
    for(Integer i : c)
      System.out.print(i + ", ");
  }
} /* Output:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
*///:~
```

因为这个示例只使用了**Collection**方法，因此任何继承自**Collection**的类的对象都可以正常工作，但是**ArrayList**是最基本的序列类型。

add()方法的名称就表明它是要将一个新元素放置到**Collection**中。但是，文档中非常仔细地叙述到：“要确保这个**Collection**包含指定的元素。”这是因为考虑到了**Set**的含义，因为在**Set**中

395 只有元素不存在的情况下才会添加。在使用**ArrayList**，或者任何种类的**List**时，**add()**总是表示“把它放进去”，因为**List**不关心是否存在重复。

所有的**Collection**都可以用foreach语法遍历，就像这里所展示的。在本章的后续部分，你将会学习到被称为“迭代器”的更灵活的概念。

练习2：(1) 修改**SimpleCollection.java**，使用**Set**来表示**c**。

练习3：(2) 修改**innerclasses/Sequence.java**，使你可以向其中添加任意数量的元素。

11.3 添加一组元素

在**java.util**包中的**Arrays**和**Collections**类中都有很多实用方法，可以在一个**Collection**中添加一组元素。**Arrays.asList()**方法接受一个数组或是一个用逗号分隔的元素列表（使用可变参数），并将其转换为一个**List**对象。**Collections.addAll()**方法接受一个**Collection**对象，以及一个数组或是一个用逗号分割的列表，将元素添加到**Collection**中。下面的示例展示了这两个方法，以及更加传统**addAll()**方法，所有**Collection**类型都包含该方法：

```
//: holding/AddingGroups.java
// Adding groups of elements to Collection objects.
import java.util.*;

public class AddingGroups {
  public static void main(String[] args) {
    Collection<Integer> collection =
      new ArrayList<Integer>(Arrays.asList(1, 2, 3, 4, 5));
    Integer[] moreInts = { 6, 7, 8, 9, 10 };
    collection.addAll(Arrays.asList(moreInts));
    // Runs significantly faster, but you can't
    // construct a Collection this way:
    Collections.addAll(collection, 11, 12, 13, 14, 15);
    Collections.addAll(collection, moreInts);
    // Produces a list "backed by" an array:
    List<Integer> list = Arrays.asList(16, 17, 18, 19, 20);
    list.set(1, 99); // OK -- modify an element
    // list.add(21); // Runtime error because the
                     // underlying array cannot be resized.
  }
} ///:~
```

396

Collection的构造器可以接受另一个**Collection**，用它来将自身初始化，因此你可以使用**Arrays.List()**来为这个构造器产生输入。但是，**Collection.addAll()**方法运行起来要快得多，而且构建一个不包含元素的**Collection**，然后调用**Collections.addAll()**这种方式很方便，因此它是首选方式。

Collection.addAll()成员方法只能接受另一个**Collection**对象作为参数，因此它不如**Arrays.asList()**或**Collections.addAll()**灵活，这两个方法使用的都是可变参数列表。

你也可以直接使用**Arrays.asList()**的输出，将其当作**List**，但是在这种情况下，其底层表示的是数组，因此不能调整尺寸。如果你试图用**add()**或**delete()**方法在这种列表中添加或删除元素，就有可能会引发去改变数组尺寸的尝试，因此你将在运行时获得“Unsupported Operation（不支持的操作）”错误。

Arrays.asList()方法的限制是它对所产生的**List**的类型做出了最理想的假设，而并没有注意你对它会赋予什么样的类型。有时这就会引发问题：

```
//: holding/AsListInference.java
// Arrays.asList() makes its best guess about type.
import java.util.*;
```

```
class Snow {}
class Powder extends Snow {}
class Light extends Powder {}
class Heavy extends Powder {}
class Crusty extends Snow {}
class Slush extends Snow {}

public class AsListInference {
  public static void main(String[] args) {
    List<Snow> snow1 = Arrays.asList(
      new Crusty(), new Slush(), new Powder());

    // Won't compile:
    // List<Snow> snow2 = Arrays.asList(
    //   new Light(), new Heavy());
    // Compiler says:
    // found   : java.util.List<Powder>
    // required: java.util.List<Snow>
    // Collections.addAll() doesn't get confused:
    List<Snow> snow3 = new ArrayList<Snow>();
    Collections.addAll(snow3, new Light(), new Heavy());

    // Give a hint using an
    // explicit type argument specification:
    List<Snow> snow4 = Arrays.<Snow>asList(
      new Light(), new Heavy());
  }
} ///:~
```

397

当试图创建**snow2**时，**Arrays.asList()**中只有**Powder**类型，因此它会创建**List<Powder>**而不是**List<Snow>**，尽管**Collections.addAll()**工作的很好，因为它从第一个参数中了解到了目标类型是什么。

正如你从创建**snow4**的操作中所看到的，可以在**Arrays.asList()**中间插入一条“线索”，以告诉编译器对于由**Arrays.asList()**产生的**List**类型，实际的目标类型应该是什么。这称为*显式类型参数说明*。

正如你所见，**Map**更加复杂，并且除了用另一个**Map**之外，Java标准类库没有提供其他任何自动初始化它们的方式。

11.4 容器的打印

你必须使用**Arrays.toString()**来产生数组的可打印表示，但是打印容器无需任何帮助。下面是一个例子，这个例子中也介绍了一些基本类型的容器：

```
//: holding/PrintingContainers.java
// Containers print themselves automatically.
import java.util.*;
import static net.mindview.util.Print.*;

public class PrintingContainers {
  static Collection fill(Collection<String> collection) {
    collection.add("rat");
    collection.add("cat");
    collection.add("dog");
    collection.add("dog");
    return collection;
  }
  static Map fill(Map<String,String> map) {
    map.put("rat", "Fuzzy");
    map.put("cat", "Rags");
    map.put("dog", "Bosco");
```

398

```
    map.put("dog", "Spot");
    return map;
  }
  public static void main(String[] args) {
    print(fill(new ArrayList<String>()));
    print(fill(new LinkedList<String>()));
    print(fill(new HashSet<String>()));
    print(fill(new TreeSet<String>()));
    print(fill(new LinkedHashSet<String>()));
    print(fill(new HashMap<String,String>()));
    print(fill(new TreeMap<String,String>()));
    print(fill(new LinkedHashMap<String,String>()));
  }
} /* Output:
[rat, cat, dog, dog]
[rat, cat, dog, dog]
[dog, cat, rat]
[cat, dog, rat]
[rat, cat, dog]
{dog=Spot, cat=Rags, rat=Fuzzy}
{cat=Rags, dog=Spot, rat=Fuzzy}
{rat=Fuzzy, cat=Rags, dog=Spot}
*///:~
```

这里展示了Java容器类库中的两种主要类型，它们的区别在于容器中每个“槽”保存的元素个数。**Collection**在每个槽中只能保存一个元素。此类容器包括：**List**，它以特定的顺序保存一组元素；**Set**，元素不能重复；**Queue**，只允许在容器的一“端”插入对象，并从另外一“端”移除对象（对于本例来说，这只是另外一种观察序列的方式，因此并没有展示它）。**Map**在每个槽内保存了两个对象，即键和与之相关联的值。

查看输出会发现，默认的打印行为（使用容器提供的**toString()**方法）即可生成可读性很好的结果。**Collection**打印出来的内容用方括号括住，每个元素由逗号分隔。**Map**则用大括号括住，键与值由等号联系（键在等号左边，值在右边）。

399

第一个**fill()**方法可以作用于所有类型的**Collection**，这些类型都实现了用来添加新元素的**add()**方法。

ArrayList和**LinkedList**都是**List**类型，从输出可以看出，它们都按照被插入的顺序保存元素。两者的不同之处不仅在于执行某些类型的操作时的性能，而且**LinkedList**包含的操作也多于**ArrayList**。这些将在本章后续部分更详细地讨论。

HashSet、**TreeSet**和**LinkedHashSet**都是**Set**类型，输出显示在**Set**中，每个相同的项只有保存一次，但是输出也显示了不同的**Set**实现存储元素的方式也不同。**HashSet**使用的是相当复杂的方式来存储元素的，这种方式将在第17章中介绍，此刻你只需要知道这种技术是最快的获取元素方式，因此，存储的顺序看起来并无实际意义（通常你只会关心某事物是否是某个**Set**的成员，而不会关心它在**Set**出现的顺序）。如果存储顺序很重要，那么可以使用**TreeSet**，它按照比较结果的升序保存对象；或者使用**LinkedHashSet**，它按照被添加的顺序保存对象。

Map（也被称为关联数组）使得你可以用键来查找对象，就像一个简单的数据库。键所关联的对象称为值。使用**Map**可以将美国州名与其首府联系起来，如果想知道Ohio的首府，可以将Ohio作为键进行查找，几乎就像使用数组下标一样。正由于这种行为，对于每一个键，**Map**只接受存储一次。

Map.put(key,value)方法将增加一个值（你想要增加的对象），并将它与某个键（你用来查找这个值的对象）关联起来。**Map.get(key)**方法将产生与这个键相关联的值。上面的示例只添加了键-值对，并没有执行查找，这将在稍后展示。

注意，你不必指定（或考虑）**Map**的尺寸，因为它自己会自动地调整尺寸。**Map**还知道如

何打印自己，它会显示相关联的键和值。键和值在**Map**中的保存顺序并不是它们的插入顺序，因为**HashMap**实现使用的是一种非常快的算法来控制顺序。 400

本例使用了三种基本风格的**Map**：**HashMap**、**TreeMap**和**LinkedHashMap**。与**HashSet**一样，**HashMap**也提供了最快的查找技术，也没有按照任何明显的顺序来保存其元素。**TreeMap**按照比较结果的升序保存键，而**LinkedHashMap**则按照插入顺序保存键，同时还保留了**HashMap**的查询速度。

练习4：(3) 创建一个生成器类，它可以在每次调用其**next()**方法时，产生你由你最喜欢的电影（你可以使用Snow White或Star Wars）的字符构成的名字（作为**String**对象）。在字符列表中的电影名用完之后，循环到这个字符列表的开始处。使用这个生成器来填充数组、**ArrayList**、**LinkedList**、**HashSet**、**LinkedHashSet**和**TreeSet**，然后打印每一个容器。

11.5 List

List承诺可以将元素维护在特定的序列中。**List**接口在**Collection**的基础上添加了大量的方法，使得可以在**List**的中间插入和移除元素。

有两种类型的**List**：

- 基本的**ArrayList**，它长于随机访问元素，但是在**List**的中间插入和移除元素时较慢。
- **LinkedList**，它通过代价较低的在**List**中间进行的插入和删除操作，提供了优化的顺序访问。**LinkedList**在随机访问方面相对比较慢，但是它的特性集较**ArrayList**更大。

下面的示例通过导入**typeinfo.pets**，超前使用了第14章中的类库。这个类库包含了**Pet**类的继承层次结构，以及用于随机生成**Pet**对象的一些工具类。此时，你还不需要了解这个类库的全部内容，而只需要知道两点：(1)有一个**Pet**类，以及**Pet**的各种子类型；(2)静态的**Pets.arrayList()**方法将返回一个填充了随机选取的**Pet**对象的**ArrayList**：

```
//: holding/ListFeatures.java
import typeinfo.pets.*;
import java.util.*;
import static net.mindview.util.Print.*;
public class ListFeatures {
  public static void main(String[] args) {
    Random rand = new Random(47);
    List<Pet> pets = Pets.arrayList(7);
    print("1: " + pets);
    Hamster h = new Hamster();
    pets.add(h); // Automatically resizes
    print("2: " + pets);
    print("3: " + pets.contains(h));
    pets.remove(h); // Remove by object
    Pet p = pets.get(2);
    print("4: " +  p + " " + pets.indexOf(p));
    Pet cymric = new Cymric();
    print("5: " + pets.indexOf(cymric));
    print("6: " + pets.remove(cymric));
    // Must be the exact object:
    print("7: " + pets.remove(p));
    print("8: " + pets);
    pets.add(3, new Mouse()); // Insert at an index
    print("9: " + pets);
    List<Pet> sub = pets.subList(1, 4);
    print("subList: " + sub);
    print("10: " + pets.containsAll(sub));
    Collections.sort(sub); // In-place sort
    print("sorted subList: " + sub);
    // Order is not important in containsAll():
```

401

```
    print("11: " + pets.containsAll(sub));
    Collections.shuffle(sub, rand); // Mix it up
    print("shuffled subList: " + sub);
    print("12: " + pets.containsAll(sub));
    List<Pet> copy = new ArrayList<Pet>(pets);
    sub = Arrays.asList(pets.get(1), pets.get(4));
    print("sub: " + sub);
    copy.retainAll(sub);
    print("13: " + copy);
    copy = new ArrayList<Pet>(pets); // Get a fresh copy
    copy.remove(2); // Remove by index
    print("14: " + copy);
    copy.removeAll(sub); // Only removes exact objects
    print("15: " + copy);
    copy.set(1, new Mouse()); // Replace an element
    print("16: " + copy);
    copy.addAll(2, sub); // Insert a list in the middle
    print("17: " + copy);
    print("18: " + pets.isEmpty());
    pets.clear(); // Remove all elements
    print("19: " + pets);
    print("20: " + pets.isEmpty());
    pets.addAll(Pets.arrayList(4));
    print("21: " + pets);
    Object[] o = pets.toArray();
    print("22: " + o[3]);
    Pet[] pa = pets.toArray(new Pet[0]);
    print("23: " + pa[3].id());
  }
} /* Output:
1: [Rat, Manx, Cymric, Mutt, Pug, Cymric, Pug]
2: [Rat, Manx, Cymric, Mutt, Pug, Cymric, Pug, Hamster]
3: true
4: Cymric 2
5: -1
6: false
7: true
8: [Rat, Manx, Mutt, Pug, Cymric, Pug]
9: [Rat, Manx, Mutt, Mouse, Pug, Cymric, Pug]
subList: [Manx, Mutt, Mouse]
10: true
sorted subList: [Manx, Mouse, Mutt]
11: true
shuffled subList: [Mouse, Manx, Mutt]
12: true
sub: [Mouse, Pug]
13: [Mouse, Pug]
14: [Rat, Mouse, Mutt, Pug, Cymric, Pug]
15: [Rat, Mutt, Cymric, Pug]
16: [Rat, Mouse, Cymric, Pug]
17: [Rat, Mouse, Mouse, Pug, Cymric, Pug]
18: false
19: []
20: true
21: [Manx, Cymric, Rat, EgyptianMau]
22: EgyptianMau
23: 14
*///:~
```

402

打印行都编了号，因此输出可以与源码相关。第一行输出展示了最初的由**Pet**构成的**List**。与数组不同，**List**允许在它被创建之后添加元素、移除元素，或者自我调整尺寸。这正是它的重要价值所在：一种可修改的序列。你可以在输出行2中看到添加一个**Hamster**的结果，即对象被追加到了表尾。

403

你可以用**contains()**方法来确定某个对象是否在列表中。如果你想移除一个对象，则可以将

这个对象的引用传递给**remove()**方法。同样，如果你有一个对象的引用，则可以使用**indexOf()**来发现该对象在**List**中所处位置的索引编号，就像你在输出行4中所见一样。

当确定一个元素是否属于某个**List**，发现某个元素的索引，以及从某个**List**中移除一个元素时，都会用到**equals()**方法（它是根类**Object**的一部分）。每个**Pet**都被定义为唯一的对象，因此即使在列表中已经有两个**Cymric**，如果我再新创建一个**Cymric**，并把它传递给**indexOf()**方法，其结果仍会是−1（表示未找到它），而且尝试调用**remove()**方法来删除这个对象，也会返回**false**。对于其他的类，**equals()**的定义可能有所不同。例如，两个**String**只有在内容完全一样的情况下才会是等价的。因此为了防止意外，就必须意识到**List**的行为根据**equals()**的行为而有所变化。

在输出行7和8中，展示了对精确匹配**List**中某个对象的对象进行移除是成功的。

在**List**中间插入元素是可行的，就像你在输出行9和它前面的代码中所看到的一样。但是这带来了一个问题：对于**LinkedList**，在列表中间插入和删除都是廉价操作（在本例中，除了对列表中间进行的真正的随机访问），但是对于**ArrayList**，这可是代价高昂的操作。这是否意味着你应该永远都不要在**ArrayList**的中间插入元素，并最好是切换到**LinkedList**？不，这仅仅意味着，你应该意识到这个问题，如果你开始在某个**ArrayList**的中间执行很多插入操作，并且你的程序开始变慢，那么你应该看看你的**List**实现有可能就是罪魁祸首（发现此类瓶颈的最佳方式是使用仿真器，就像你在http://MindView.net/Books/BetterJava上的补充材料中所看到的一样）。优化是一个很棘手的问题，最好的策略就是置之不顾，直到你发现需要担心它了（尽管理解这些问题总是一种好的思路）。

404

subList()方法允许你很容易地从较大的列表中创建出一个片断，而将其结果传递给这个较大的列表的**containsAll()**方法时，很自然地会得到**true**。还有一点也很有趣，那就是我们注意到顺序并不重要，你可以在输出行11和12中看到，在**sub**上调用名字很直观的**Collections.sort()**和**Collection.shuffle()**方法，不会影响**containsAll()**的结果。**subList()**所产生的列表的幕后就是初始列表，因此，对所返回的列表的修改都会反映到初始列表中，反之亦然。

retainAll()方法是一种有效的“交集”操作，在本例中，它保留了所有同时在**copy**与**sub**中的元素。请再次注意，所产生的行为依赖于**equals()**方法。

输出行14展示了用元素的索引值来移除元素的结果。与通过对象引用来移除相比，它显得更加直观，因为在使用索引值时，不必担心**equals()**的行为。

removeAll()方法的行为也是基于**equals()**方法的。就像其名称所表示的，它将从**List**中移除在参数**List**中的所有元素。

set()方法的命名显得很不合时宜，因为它与**Set**类存在潜在的冲突。在此处，replace可能会显得更适合，因为它的功能是在指定的索引处（第一个参数），用第二个参数替换整个位置的元素。

输出行17表明，对于**List**，有一个重载的**addAll()**方法使得我们可以在初始**List**的中间插入新的列表，而不仅仅只能用**Collection**中的**addAll()**方法将其追加到表尾。

输出行18-20展示了**isEmpty()**和**clear()**方法的效果。

输出行22-23展示了你可以如何通过使用**toArray()**方法，将任意的**Collection**转换为一个数组。这是一个重载方法，其无参数版本返回的是**Object**数组，但是如果你向这个重载版本传递目标类型的数据，那么它将产生指定类型的数据（假设它能通过类型检查）。如果参数数组太小，存放不下**List**中的所有元素（就像本例一样），**toArray()**方法将创建一个具有合适尺寸的数组。**Pet**对象具有一个**id()**方法，如你所见，可以在所产生的数组中的对象上调用这个方法。

405

练习5：(3) 修改**ListFeatures.java**，让它使用**Integer**（记住自动包装机制！）而不是**Pet**，并解释在结果上有何不同。

练习6：(2) 修改**ListFeatures.java**，让它使用**String**而不是**Pet**，并解释在结果上有何不同。

练习7：(3) 创建一个类，然后创建一个用你的类的对象进行过初始化的数组。通过使用**subList()**方法，创建你的**List**的子集，然后在你的**List**中移除这个子集。

11.6 迭代器

任何容器类，都必须有某种方式可以插入元素并将它们再次取回。毕竟，持有事物是容器最基本的工作。对于**List**，**add()**是插入元素的方法之一，而**get()**是取出元素的方法之一。

如果从更高层的角度思考，会发现这里有个缺点：要使用容器，必须对容器的确切类型编程。初看起来这没什么不好，但是考虑下面的情况：如果原本是对着**List**编码的，但是后来发现如果能够把相同的代码应用于**Set**，将会显得非常方便，此时应该怎么做？或者打算从头开始编写通用的代码，它们只是使用容器，不知道或者说不关心容器的类型，那么如何才能不重写代码就可以应用于不同类型的容器？

迭代器（也是一种设计模式）的概念可以用于达成此目的。迭代器是一个对象，它的工作是遍历并选择序列中的对象，而客户端程序员不必知道或关心该序列底层的结构。此外，迭代器通常被称为*轻量级对象*：创建它的代价小。因此，经常可以见到对迭代器有些奇怪的限制；例如，Java的**Iterator**只能单向移动，这个**Iterator**只能用来：

1) 使用方法**iterator()**要求容器返回一个**Iterator**。**Iterator**将准备好返回序列的第一个元素。
2) 使用**next()**获得序列中的下一个元素。
3) 使用**hasNext()**检查序列中是否还有元素。
4) 使用**remove()**将迭代器新近返回的元素删除。

406

为了观察它的工作方式，让我们再次使用在第14章中的**Pet**工具：

```
//: holding/SimpleIteration.java
import typeinfo.pets.*;
import java.util.*;

public class SimpleIteration {
  public static void main(String[] args) {
    List<Pet> pets = Pets.arrayList(12);
    Iterator<Pet> it = pets.iterator();
    while(it.hasNext()) {
      Pet p = it.next();
      System.out.print(p.id() + ":" + p + " ");
    }
    System.out.println();
    // A simpler approach, when possible:
    for(Pet p : pets)
      System.out.print(p.id() + ":" + p + " ");
    System.out.println();
    // An Iterator can also remove elements:
    it = pets.iterator();
    for(int i = 0; i < 6; i++) {
      it.next();
      it.remove();
    }
    System.out.println(pets);
  }
} /* Output:
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
8:Cymric 9:Rat 10:EgyptianMau 11:Hamster
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
8:Cymric 9:Rat 10:EgyptianMau 11:Hamster
[Pug, Manx, Cymric, Rat, EgyptianMau, Hamster]
*///:~
```

有了**Iterator**就不必为容器中元素的数量操心了，那是由**hasNext()**和**next()**关心的事情。

如果你只是向前遍历**List**，并不打算修改**List**对象本身，那么你可以看到**foreach**语法会显得更加简洁。

407

Iterator还可以移除由**next()**产生的最后一个元素，这意味着在调用**remove()**之前必须先调用**next()**[⊖]。

接受对象容器并传递它，从而在每个对象上都执行操作，这种思想十分强大，并且贯穿于本书。

现在考虑创建一个**display()**方法，它不必知晓容器的确切类型：

```
//: holding/CrossContainerIteration.java
import typeinfo.pets.*;
import java.util.*;

public class CrossContainerIteration {
  public static void display(Iterator<Pet> it) {
    while(it.hasNext()) {
      Pet p = it.next();
      System.out.print(p.id() + ":" + p + " ");
    }
    System.out.println();
  }
  public static void main(String[] args) {
    ArrayList<Pet> pets = Pets.arrayList(8);
    LinkedList<Pet> petsLL = new LinkedList<Pet>(pets);
    HashSet<Pet> petsHS = new HashSet<Pet>(pets);
    TreeSet<Pet> petsTS = new TreeSet<Pet>(pets);
    display(pets.iterator());
    display(petsLL.iterator());
    display(petsHS.iterator());
    display(petsTS.iterator());
  }
} /* Output:
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
4:Pug 6:Pug 3:Mutt 1:Manx 5:Cymric 7:Manx 2:Cymric 0:Rat
5:Cymric 2:Cymric 7:Manx 1:Manx 3:Mutt 6:Pug 4:Pug 0:Rat
*///:~
```

408

请注意，**display()**方法不包含任何有关它所遍历的序列的类型信息，而这也展示了**Iterator**的真正威力：能够将遍历序列的操作与序列底层的结构分离。正由于此，我们有时会说：迭代器统一了对容器的访问方式。

练习8：(1) 修改练习1，以便调用**hop()**时使用**Iterator**遍历**List**。

练习9：(4) 修改**innerclasses/Sequence.java**，使得在**Sequence**中，用**Iterator**取代**Selector**。

练习10：(2) 修改第8章中的练习9，使其使用一个**ArrayList**来存放**Rodents**，并使用一个**Iterator**来访问**Rodent**序列。

练习11：(2) 写一个方法，使用**Iterator**遍历**Collection**，并打印容器中每个对象的**toString()**。填充各种类型的**Collection**，然后对其使用此方法。

11.6.1 ListIterator

ListIterator是一个更加强大的**Iterator**的子类型，它只能用于各种**List**类的访问。尽管**Iterator**只能向前移动，但是**ListIterator**可以双向移动。它还可以产生相对于迭代器在列表中指

⊖ **remove()**是所谓的“可选”方法（还有一些其他的这种方法），即不是所有的**Iterator**实现都必须实现该方法。这个问题将在第17章中介绍。但是，标准Java类库实现了**remove()**，因此直至第17章之前，你都不用担心这个问题。

向的当前位置的前一个和后一个元素的索引，并且可以使用**set()**方法替换它访问过的最后一个元素。你可以通过调用**listIterator()**方法产生一个指向**List**开始处的**ListIterator**，并且还可以通过调用**listIterator(n)**方法创建一个一开始就指向列表索引为**n**的元素处的**ListIterator**。下面的示例演示了所有这些能力：

```
//: holding/ListIteration.java
import typeinfo.pets.*;
import java.util.*;

public class ListIteration {
  public static void main(String[] args) {
    List<Pet> pets = Pets.arrayList(8);
    ListIterator<Pet> it = pets.listIterator();
    while(it.hasNext())
      System.out.print(it.next() + ", " + it.nextIndex() +
        ", " + it.previousIndex() + "; ");
    System.out.println();
    // Backwards:
    while(it.hasPrevious())
      System.out.print(it.previous().id() + " ");
    System.out.println();
    System.out.println(pets);
    it = pets.listIterator(3);
    while(it.hasNext()) {
      it.next();
      it.set(Pets.randomPet());
    }
    System.out.println(pets);
  }
} /* Output:
Rat, 1, 0; Manx, 2, 1; Cymric, 3, 2; Mutt, 4, 3; Pug, 5, 4;
Cymric, 6, 5; Pug, 7, 6; Manx, 8, 7;
7 6 5 4 3 2 1 0
[Rat, Manx, Cymric, Mutt, Pug, Cymric, Pug, Manx]
[Rat, Manx, Cymric, Cymric, Rat, EgyptianMau, Hamster,
EgyptianMau]
*///:~
```

409

Pet.randomPet()方法用来替换在列表中从位置3开始向前的所有**Pet**对象。

练习12：(3) 创建并组装一个**List<Integer>**，然后创建第二个具有相同尺寸的**List<Integer>**，并使用**ListIterator**读取第一个**List**中的元素，然后再将它们以反序插入到第二个列表中（你可能想探索该问题的各种不同的解决之道）。

11.7 LinkedList

LinkedList也像**ArrayList**一样实现了基本的**List**接口，但是它执行某些操作（在**List**的中间插入和移除）时比**ArrayList**更高效，但在随机访问操作方面却要逊色一些。

LinkedList还添加了可以使其用作栈、队列或双端队列的方法。

这些方法中有些彼此之间只是名称有些差异，或者只存在些许差异，以使得这些名字在特定用法的上下文环境中更加适用（特别是在**Queue**中）。例如，**getFirst()**和**element()**完全一样，它们都返回列表的头（第一个元素），而并不移除它，如果**List**为空，则抛出**NoSuchElementException**。**peek()**方法与这两个方式只是稍有差异，它在列表为空时返回**null**。

410

removeFirst()与**remove()**也是完全一样的，它们移除并返回列表的头，而在列表为空时抛出**NoSuchElementException**。**poll()**稍有差异，它在列表为空时返回**null**。

addFirst()与**add()**和**addLast()**相同，它们都将某个元素插入到列表的尾（端）部。

removeLast()移除并返回列表的最后一个元素。

下面的示例展示了这些特性之间基本的相同性和差异性，它重复地执行**ListFeatures.java**中所示的行为：

```
//: holding/LinkedListFeatures.java
import typeinfo.pets.*;
import java.util.*;
import static net.mindview.util.Print.*;

public class LinkedListFeatures {
  public static void main(String[] args) {
    LinkedList<Pet> pets =
      new LinkedList<Pet>(Pets.arrayList(5));
    print(pets);
    // Identical:
    print("pets.getFirst(): " + pets.getFirst());
    print("pets.element(): " + pets.element());
    // Only differs in empty-list behavior:
    print("pets.peek(): " + pets.peek());
    // Identical; remove and return the first element:
    print("pets.remove(): " + pets.remove());
    print("pets.removeFirst(): " + pets.removeFirst());
    // Only differs in empty-list behavior:
    print("pets.poll(): " + pets.poll());
    print(pets);
    pets.addFirst(new Rat());
    print("After addFirst(): " + pets);
    pets.offer(Pets.randomPet());
    print("After offer(): " + pets);
    pets.add(Pets.randomPet());
    print("After add(): " + pets);
    pets.addLast(new Hamster());
    print("After addLast(): " + pets);
    print("pets.removeLast(): " + pets.removeLast());
  }
} /* Output:
[Rat, Manx, Cymric, Mutt, Pug]
pets.getFirst(): Rat
pets.element(): Rat
pets.peek(): Rat
pets.remove(): Rat
pets.removeFirst(): Manx
pets.poll(): Cymric
[Mutt, Pug]
After addFirst(): [Rat, Mutt, Pug]
After offer(): [Rat, Mutt, Pug, Cymric]
After add(): [Rat, Mutt, Pug, Cymric, Pug]
After addLast(): [Rat, Mutt, Pug, Cymric, Pug, Hamster]
pets.removeLast(): Hamster
*///:~
```

411

Pets.arrayList()的结果交给了**LinkedList**的构造器，以便使用它来组装**LinkedList**。如果你浏览一下**Queue**接口就会发现，它在**LinkedList**的基础上添加了**element()**、**offer()**、**peek()**、**poll()**和**remove()**方法，以使其可以成为一个**Queue**的实现。**Queue**的完整示例将在本章稍后给出。

练习13：(3) 在**innerclasses/GreenhouseController.java**示例中，**Controller**类使用的是**ArrayList**，修改代码，用**LinkedList**替换之，并使用**Iterator**来循环遍历事件集。

练习14：(3) 创建一个空的**LinkedList<Integer>**，通过使用**ListIterator**，将若干个**Integer**插入这个**List**中，插入时，总是将它们插入到List的中间。

11.8 Stack

“栈”通常是指“后进先出”（LIFO）的容器。有时栈也被称为*叠加栈*，因为最后“压入”

栈的元素，第一个“弹出”栈。经常用来类比栈的事物是装有弹簧的储放器中的自助餐托盘，最后装入的托盘总是最先拿出使用的。

412

LinkedList具有能够直接实现栈的所有功能的方法，因此可以直接将**LinkedList**作为栈使用。不过，有时一个真正的“栈”更能把事情讲清楚：

```
//: net/mindview/util/Stack.java
// Making a stack from a LinkedList.
package net.mindview.util;
import java.util.LinkedList;

public class Stack<T> {
  private LinkedList<T> storage = new LinkedList<T>();
  public void push(T v) { storage.addFirst(v); }
  public T peek() { return storage.getFirst(); }
  public T pop() { return storage.removeFirst(); }
  public boolean empty() { return storage.isEmpty(); }
  public String toString() { return storage.toString(); }
} ///:~
```

这里通过使用范型，引入了在栈的类定义中最简单的可行示例。类名之后的**<T>**告诉编译器这将是一个参数化类型，而其中的类型参数，即在类被使用时将会被实际类型替换的参数，就是**T**。大体上，这个类是在声明“我们在定义一个可以持有**T**类型对象的**Stack**。”**Stack**是用**LinkedList**实现的，而**LinkedList**也被告知它将持有**T**类型对象。注意，**push()**接受的是**T**类型的对象，而**peek()**和**pop()**将返回**T**类型的对象。**peek()**方法将提供栈顶元素，但是并不将其从栈顶移除，而**pop()**将移除并返回栈顶元素。

如果你只需要栈的行为，这里使用继承就不合适了，因为这样会产生具有**LinkedList**的其他所有方法的类（就象你将在第17章中所看到的，**Java1.0**的设计者在创建**java.util.Stack**时，就犯了这个错误）。

下面演示了这个新的**Stack**类：

```
//: holding/StackTest.java
import net.mindview.util.*;

public class StackTest {
  public static void main(String[] args) {
    Stack<String> stack = new Stack<String>();
    for(String s : "My dog has fleas".split(" "))
      stack.push(s);
    while(!stack.empty())
      System.out.print(stack.pop() + " ");
  }
} /* Output:
fleas has dog My
*///:~
```

413

如果你想在自己的代码中使用这个**Stack**类，当你在创建其实例时，就需要完整指定包名，或者更改这个类的名称；否则，就有可能与**java.util**包中的**Stack**发生冲突。例如，如果我们在上面的例子中导入**java.util.***，那么就必须使用包名以防止冲突：

```
//: holding/StackCollision.java
import net.mindview.util.*;

public class StackCollision {
  public static void main(String[] args) {
    net.mindview.util.Stack<String> stack =
      new net.mindview.util.Stack<String>();
    for(String s : "My dog has fleas".split(" "))
      stack.push(s);
```

```
    while(!stack.empty())
      System.out.print(stack.pop() + " ");
    System.out.println();
    java.util.Stack<String> stack2 =
      new java.util.Stack<String>();
    for(String s : "My dog has fleas".split(" "))
      stack2.push(s);
    while(!stack2.empty())
      System.out.print(stack2.pop() + " ");
  }
} /* Output:
fleas has dog My
fleas has dog My
*///:~
```

这两个**Stack**具有相同的接口，但是在**java.util**中没有任何公共的**Stack**接口，这可能是因为在**Java1.0**中的设计欠佳的最初的**java.util.Stack**类占用了这个名字。尽管已经有了**java.util.Stack**，但是**LinkedList**可以产生更好的**Stack**，因此**net.mindview.util.Stack**所采用的方式更是可取的。 414

你还可以通过显式的导入来控制对“首选”**Stack**实现的选择：

```
import net.mindview.util.Stack;
```

现在，任何对**Stack**的引用都将选择**net.mindview.util**版本，而在选择**java.util.Stack**时，必须使用全限定名称。

练习15：(4) 栈在编程语言中经常用来对表达式求值。请使用**net.mndview.util.Stack**对下面的表达式求值，其中“+”表示“将后面的字母压进栈”，而“–”表示“弹出栈顶字母并打印它”：

“+U+n+c---+e+r+t---+a-+i-+n+t+y---+-+r+u--+l+e+s---”

11.9 Set

Set不保存重复的元素（至于如何判断元素相同则较为复杂，稍后便会看到）。如果你试图将相同对象的多个实例添加到**Set**中，那么它就会阻止这种重复现象。**Set**中最常被使用的是测试归属性，你可以很容易地询问某个对象是否在某个**Set**中。正因如此，查找就成为了**Set**中最重要的操作，因此你通常都会选择一个**HashSet**的实现，它专门对快速查找进行了优化。

Set具有与**Collection**完全一样的接口，因此没有任何额外的功能，不像前面有两个不同的**List**。实际上**Set**就是**Collection**，只是行为不同。（这是继承与多态思想的典型应用：表现不同的行为。）**Set**是基于对象的值来确定归属性的，而更加复杂的问题我们将在第17章中介绍。

下面是使用存放**Integer**对象的**HashSet**的示例：

```
//: holding/SetOfInteger.java
import java.util.*;

public class SetOfInteger {
  public static void main(String[] args) {
    Random rand = new Random(47);
    Set<Integer> intset = new HashSet<Integer>();
    for(int i = 0; i < 10000; i++)
      intset.add(rand.nextInt(30));
    System.out.println(intset);
  }
} /* Output:
[15, 8, 23, 16, 7, 22, 9, 21, 6, 1, 29, 14, 24, 4, 19, 26,
11, 18, 3, 12, 27, 17, 2, 13, 28, 20, 25, 10, 5, 0]
*///:~
```

415

在0到29之间的10000个随机数被添加到了**Set**中，因此你可以想象，每一个数都重复了许多次。但是你可以看到，每一个数只有一个实例出现在结果中。

你还可以注意到，输出的顺序没有任何规律可循，这是因为出于速度原因的考虑，**HashSet**使用了散列——散列将在第17章中介绍。**HashSet**所维护的顺序与**TreeSet**或**LinkedHashSet**都不同，因为它们的实现具有不同的元素存储方式。**TreeSet**将元素存储在红-黑树数据结构中，而**HashSet**使用的是散列函数。**LinkedHashList**因为查询速度的原因也使用了散列，但是看起来它使用了链表来维护元素的插入顺序。

如果你想对结果排序，一种方式是使用**TreeSet**来代替**HashSet**：

```
//: holding/SortedSetOfInteger.java
import java.util.*;

public class SortedSetOfInteger {
  public static void main(String[] args) {
    Random rand = new Random(47);
    SortedSet<Integer> intset = new TreeSet<Integer>();
    for(int i = 0; i < 10000; i++)
      intset.add(rand.nextInt(30));
    System.out.println(intset);
  }
} /* Output:
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]
*///:~
```

你将会执行的最常见的操作之一，就是使用**contains()**测试**Set**的归属性，但是还有很多操作会让你想起在上小学时所教授的文氏图（译者注：用圆表示集与集之间关系的图）：

416

```
//: holding/SetOperations.java
import java.util.*;
import static net.mindview.util.Print.*;

public class SetOperations {
  public static void main(String[] args) {
    Set<String> set1 = new HashSet<String>();
    Collections.addAll(set1,
      "A B C D E F G H I J K L".split(" "));
    set1.add("M");
    print("H: " + set1.contains("H"));
    print("N: " + set1.contains("N"));
    Set<String> set2 = new HashSet<String>();
    Collections.addAll(set2, "H I J K L".split(" "));
    print("set2 in set1: " + set1.containsAll(set2));
    set1.remove("H");
    print("set1: " + set1);
    print("set2 in set1: " + set1.containsAll(set2));
    set1.removeAll(set2);
    print("set2 removed from set1: " + set1);
    Collections.addAll(set1, "X Y Z".split(" "));
    print("'X Y Z' added to set1: " + set1);
  }
} /* Output:
H: true
N: false
set2 in set1: true
set1: [D, K, C, B, L, G, I, M, A, F, J, E]
set2 in set1: false
set2 removed from set1: [D, C, B, G, M, A, F, E]
'X Y Z' added to set1: [Z, D, C, B, G, M, A, F, Y, X, E]
*///:~
```

这些方法名都是自解释型的，而有几个方法可以在JDK文档中找到。

能够产生每个元素都唯一的列表是相当有用的功能。例如，在你想要列出在上面的**SetOperations.java**文件中所有的单词的时候。通过使用本书稍后将要介绍的**net.mindview.TextFile**工具，可以打开一个文件，并将其读入一个**Set**中：

```
//: holding/UniqueWords.java
import java.util.*;
import net.mindview.util.*;

public class UniqueWords {
  public static void main(String[] args) {
    Set<String> words = new TreeSet<String>(
      new TextFile("SetOperations.java", "\\W+"));
    System.out.println(words);
  }
} /* Output:
[A, B, C, Collections, D, E, F, G, H, HashSet, I, J, K, L,
M, N, Output, Print, Set, SetOperations, String, X, Y, Z,
add, addAll, added, args, class, contains, containsAll,
false, from, holding, import, in, java, main, mindview,
net, new, print, public, remove, removeAll, removed, set1,
set2, split, static, to, true, util, void]
*///:~
```

417

TextFile继承自**List<String>**，其构造器将打开文件，并根据正则表达式“\\W+”将其断开为单词，这个正则表达式表示“一个或多个字母”（正则表达式将在第13章中介绍）。所产生的结果传递给了**TreeSet**的构造器，它将把**List**中的内容添加到自身中。由于它是**TreeSet**，因此其结果是排序的。在本例中，排序是按字典序进行的，因此大写和小写字母被划分到了不同的组中。如果你想要按照字母序排序，那么可以向**TreeSet**的构造器传入**String.CASE_INSENTIVE_ORDER**比较器（比较器就是建立排序顺序的对象）：

```
//: holding/UniqueWordsAlphabetic.java
// Producing an alphabetic listing.
import java.util.*;
import net.mindview.util.*;

public class UniqueWordsAlphabetic {
  public static void main(String[] args) {
    Set<String> words =
      new TreeSet<String>(String.CASE_INSENSITIVE_ORDER);
    words.addAll(
      new TextFile("SetOperations.java", "\\W+"));
    System.out.println(words);
  }
} /* Output:
[A, add, addAll, added, args, B, C, class, Collections,
contains, containsAll, D, E, F, false, from, G, H, HashSet,
holding, I, import, in, J, java, K, L, M, main, mindview,
N, net, new, Output, Print, public, remove, removeAll,
removed, Set, set1, set2, SetOperations, split, static,
String, to, true, util, void, X, Y, Z]
*///:~
```

418

Comparator比较器将在第16章详细介绍。

练习16：(5) 创建一个元音字母**Set**。对**UniqueWords.java**操作，计数并显示在每一个输入单词中的元音字母数量，并显示输入文件中的所有元音字母的数量总和。

11.10 Map

将对象映射到其他对象的能力是一种解决编程问题的杀手锏。例如，考虑一个程序，它将用来检查Java的**Random**类的随机性。理想状态下，**Random**可以将产生理想的数字分布，但要想测试它，则需要生成大量的随机数，并对落入各种不同范围的数字进行计数。**Map**可以很容易地解决该问题。在本例中，键是由**Random**产生的数字，而值是该数字出现的次数：

```
//: holding/Statistics.java
// Simple demonstration of HashMap.
import java.util.*;

public class Statistics {
  public static void main(String[] args) {
    Random rand = new Random(47);
    Map<Integer,Integer> m =
      new HashMap<Integer,Integer>();
    for(int i = 0; i < 10000; i++) {
      // Produce a number between 0 and 20:
      int r = rand.nextInt(20);
      Integer freq = m.get(r);
      m.put(r, freq == null ? 1 : freq + 1);
    }
    System.out.println(m);
  }
} /* Output:
{15=497, 4=481, 19=464, 8=468, 11=531, 16=533, 18=478,
3=508, 7=471, 12=521, 17=509, 2=489, 13=506, 9=549, 6=519,
1=502, 14=477, 10=513, 5=503, 0=481}
*///:~
```

在**main()**中，自动包装机制将随机生成的**int**转换为**HashMap**可以使用的**Integer**引用（不能使用基本类型的容器）。如果键不在容器中，**get()**方法将返回**null**（这表示该数字第一次被找到）。否则，**get()**方法将产生与该键相关联的**Integer**值，然后这个值被递增（自动包装机制再次简化了表达式，但是确实发生了对**Integer**的包装和拆包）。

419

下面的示例允许你使用一个**String**描述来查找**Pet**，它还展示了你可以使用怎样的方法通过使用**containsKey()**和**containsValue()**来测试一个**Map**，以便查看它是否包含某个键或某个值。

```
//: holding/PetMap.java
import typeinfo.pets.*;
import java.util.*;
import static net.mindview.util.Print.*;

public class PetMap {
  public static void main(String[] args) {
    Map<String,Pet> petMap = new HashMap<String,Pet>();
    petMap.put("My Cat", new Cat("Molly"));
    petMap.put("My Dog", new Dog("Ginger"));
    petMap.put("My Hamster", new Hamster("Bosco"));
    print(petMap);
    Pet dog = petMap.get("My Dog");
    print(dog);
    print(petMap.containsKey("My Dog"));
    print(petMap.containsValue(dog));
  }
} /* Output:
{My Cat=Cat Molly, My Hamster=Hamster Bosco, My Dog=Dog
Ginger}
Dog Ginger
true
true
*///:~
```

Map与数组和其他的**Collection**一样，可以很容易地扩展到多维，而我们只需将其值设置为**Map**（这些**Map**的值可以是其他容器，甚至是其他**Map**）。因此，我们能够很容易地将容器组合起来从而快速地生成强大的数据结构。例如，假设你正在跟踪拥有多个宠物的人，你所需只是一个**Map<Person, List<Pet>>**：

```
//: holding/MapOfList.java
package holding;
```

420

```
import typeinfo.pets.*;
import java.util.*;
import static net.mindview.util.Print.*;

public class MapOfList {
  public static Map<Person, List<? extends Pet>>
    petPeople = new HashMap<Person, List<? extends Pet>>();
  static {
    petPeople.put(new Person("Dawn"),
      Arrays.asList(new Cymric("Molly"),new Mutt("Spot")));
    petPeople.put(new Person("Kate"),
      Arrays.asList(new Cat("Shackleton"),
        new Cat("Elsie May"), new Dog("Margrett")));
    petPeople.put(new Person("Marilyn"),
      Arrays.asList(
       new Pug("Louie aka Louis Snorkelstein Dupree"),
       new Cat("Stanford aka Stinky el Negro"),
       new Cat("Pinkola")));
    petPeople.put(new Person("Luke"),
      Arrays.asList(new Rat("Fuzzy"), new Rat("Fizzy")));
    petPeople.put(new Person("Isaac"),
      Arrays.asList(new Rat("Freckly")));
  }
  public static void main(String[] args) {
    print("People: " + petPeople.keySet());
    print("Pets: " + petPeople.values());
    for(Person person : petPeople.keySet()) {
      print(person + " has:");
      for(Pet pet : petPeople.get(person))
        print("    " + pet);
    }
  }
} /* Output:
People: [Person Luke, Person Marilyn, Person Isaac, Person
Dawn, Person Kate]
Pets: [[Rat Fuzzy, Rat Fizzy], [Pug Louie aka Louis
Snorkelstein Dupree, Cat Stanford aka Stinky el Negro, Cat
Pinkola], [Rat Freckly], [Cymric Molly, Mutt Spot], [Cat
Shackleton, Cat Elsie May, Dog Margrett]]
Person Luke has:
    Rat Fuzzy
    Rat Fizzy
Person Marilyn has:
    Pug Louie aka Louis Snorkelstein Dupree
    Cat Stanford aka Stinky el Negro
    Cat Pinkola
Person Isaac has:
    Rat Freckly
Person Dawn has:
    Cymric Molly
    Mutt Spot
Person Kate has:
    Cat Shackleton
    Cat Elsie May
    Dog Margrett
*///:~
```

421

Map可以返回它的键的**Set**，它的值的**Collection**，或者它的键值对的**Set**。**keySet()**方法产生了由在**petPeople**中的所有键组成的**Set**，它在foreach语句中被用来迭代遍历该**Map**。

练习17：(2) 使用练习1中的**Gerbil**类，将其放入**Map**中，将每个**Gerbil**的名字（例如Fuzzy或Spot）**String**（键）与每个**Gerbil**（值）关联起来。为**keySet()**获取**Iterator**，使用它遍历**Map**，针对每个“键”查询**Gerbil**，然后打印出“键”，并让**gerbil**执行**hop()**。

练习18：(3) 用键值对填充一个**HashMap**。打印结果，通过散列码来展示其排序。抽取这些

键值对，按照键进行排序，并将结果置于一个**LinkedHashMap**中。展示其所维护的插入顺序。

练习19：(2) 使用**HashSet**和**LinkedHashSet**重复前一个练习。

练习20：(3) 修改练习16，使得你可以跟踪每一个元音字母出现的次数。

练习21：(3) 通过使用**Map<String.Integer>**，遵循**UniqueWords.java**的形式来创建一个程序，它可以对一个文件中出现的单词计数。使用带有第二个参数**String.CASE_INSENSITIVE_ORDER**的**Collections.sort()**方法对结果进行排序（将产生字母序），然后显示结果。

422

练习22：(5) 修改前一个练习，使其用一个包含有一个**String**域和一个计数域的类来存储每一个不同的单词，并使用一个由这些对象构成的**Set**来维护单词列表。

练习23：(4) 从**Statistics.java**开始，写一个程序，让它重复做测试，观察是否某个数字比别的数字出现的次数多。

练习24：(2) 使用**String**“键”和你选择的对象填充**LinkedHashMap**。然后从中提取键值对，以键排序，然后重新插入此**Map**。

练习25：(3) 创建一个**Map<String.ArrayList<Integer>>**，使用**net.mindview.TextFile**来打开一个文本文件，并一次读入一个单词（使用“\\W+”作为**TextFile**构造器的第二个参数）。在读入单词时对它们进行计数，并且对于文件中的每一个单词，都在**ArrayList<Integer>**中记录下与这个词相关联的单词计数。实际上，它记录的是该单词在文件中被发现的位置。

练习26：(4) 拿到前一个练习中所产生的**Map**，并按照它们在最初的文件中出现的顺序重新创建单词顺序。

11.11 Queue

队列是一个典型的先进先出（FIFO）的容器。即从容器的一端放入事物，从另一端取出，并且事物放入容器的顺序与取出的顺序是相同的。队列常被当作一种可靠的将对象从程序的某个区域传输到另一个区域的途径。队列在并发编程中特别重要，就像你将在第21章中所看到的，因为它们可以安全地将对象从一个任务传输给另一个任务。

LinkedList提供了方法以支持队列的行为，并且它实现了**Queue**接口，因此**LinkedList**可以用作**Queue**的一种实现。通过将**LinkedList**向上转型为**Queue**，下面的示例使用了在**Queue**接口中与**Queue**相关的方法：

```
//: holding/QueueDemo.java
// Upcasting to a Queue from a LinkedList.
import java.util.*;

public class QueueDemo {
  public static void printQ(Queue queue) {
    while(queue.peek() != null)
      System.out.print(queue.remove() + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    Queue<Integer> queue = new LinkedList<Integer>();
    Random rand = new Random(47);
    for(int i = 0; i < 10; i++)
      queue.offer(rand.nextInt(i + 10));
    printQ(queue);
    Queue<Character> qc = new LinkedList<Character>();
    for(char c : "Brontosaurus".toCharArray())
      qc.offer(c);
    printQ(qc);
  }
} /* Output:
```

423

```
8 1 1 1 5 14 3 1 0 1
B r o n t o s a u r u s
*///:~
```

offer()方法是与**Queue**相关的方法之一，它在允许的情况下，将一个元素插入到队尾，或者返回**false**。**peek()**和**element()**都将在不移除的情况下返回队头，但是**peek()**方法在队列为空时返回**null**，而**element()**会抛出**NoSuchElementException**异常。**poll()**和**remove()**方法将移除并返回队头，但是**poll()**在队列为空时返回**null**，而**remove()**会抛出**NoSuchElementException**异常。

自动包装机制会自动地将**nextInt()**方法的**int**结果转换为**queue**所需的**Integer**对象，将**char c**转换为**qc**所需的**Character**对象。**Queue**接口窄化了对**LinkedList**的方法的访问权限，以使得只有恰当的方法才可以使用，因此，你能够访问的**LinkedList**的方法会变少（这里你实际上可以将**queue**转型回**LinkedList**，但是至少我们不鼓励这么做）。

注意，与Queue相关的方法提供了完整而独立的功能。即，对于**Queue**所继承的**Collection**，在不需要使用它的任何方法的情况下，就可以拥有一个可用的**Queue**。

练习27：(2) 写一个称为**Command**的类，它包含一个**String**域和一个显示该**String**的**operation()**方法。写第二个类，它具有一个使用**Command**对象来填充一个**Queue**并返回这个对象的方法。将填充后的**Queue**传递给第三个类的一个方法，该方法消耗掉**Queue**中的对象，并调用它们的**operation()**方法。

424

11.11.1 PriorityQueue

先进先出描述了最典型的队列规则。队列规则是指在给定一组队列中的元素的情况下，确定下一个弹出队列的元素的规则。先进先出声明的是下一个元素应该是等待时间最长的元素。

优先级队列声明下一个弹出元素是最需要的元素（具有最高的优先级）。例如，在飞机场，当飞机临近起飞时，这架飞机的乘客可以在办理登机手续时排到队头。如果构建了一个消息系统，某些消息比其他消息更重要，因而应该更快地得到处理，那么它们何时得到处理就与它们何时到达无关。**PriorityQueue**添加到Java SE5中，是为了提供这种行为的一种自动实现。

当你在**PriorityQueue**上调用**offer()**方法来插入一个对象时，这个对象会在队列中被排序[⊖]。默认的排序将使用对象在队列中的自然顺序，但是你可以通过提供自己的**Comparator**来修改这个顺序。**PriorityQueue**可以确保当你调用**peek()**、**poll()**和**remove()**方法时，获取的元素将是队列中优先级最高的元素。

让**PriorityQueue**与**Integer**、**String**和**Character**这样的内置类型一起工作易如反掌。在下面的示例中，第一个值集与前一个示例中的随机值相同，因此你可以看到它们从**PriorityQueue**中弹出的顺序与前一个示例不同：

```
//: holding/PriorityQueueDemo.java
import java.util.*;

public class PriorityQueueDemo {
  public static void main(String[] args) {
    PriorityQueue<Integer> priorityQueue =
      new PriorityQueue<Integer>();
    Random rand = new Random(47);
    for(int i = 0; i < 10; i++)
      priorityQueue.offer(rand.nextInt(i + 10));

    QueueDemo.printQ(priorityQueue);
```

425

⊖ 这实际上依赖于具体实现。优先级队列算法通常会在插入时排序（维护一个堆），但是它们也可能在移除时选择最重要的元素。如果对象的优先级在它在队列中等待时可以进行修改，那么算法的选择就显得很重要了。

```
    List<Integer> ints = Arrays.asList(25, 22, 20,
      18, 14, 9, 3, 1, 1, 2, 3, 9, 14, 18, 21, 23, 25);
    priorityQueue = new PriorityQueue<Integer>(ints);
    QueueDemo.printQ(priorityQueue);
    priorityQueue = new PriorityQueue<Integer>(
        ints.size(), Collections.reverseOrder());
    priorityQueue.addAll(ints);
    QueueDemo.printQ(priorityQueue);

    String fact = "EDUCATION SHOULD ESCHEW OBFUSCATION";
    List<String> strings = Arrays.asList(fact.split(""));
    PriorityQueue<String> stringPQ =
      new PriorityQueue<String>(strings);
    QueueDemo.printQ(stringPQ);
    stringPQ = new PriorityQueue<String>(
      strings.size(), Collections.reverseOrder());
    stringPQ.addAll(strings);
    QueueDemo.printQ(stringPQ);

    Set<Character> charSet = new HashSet<Character>();
    for(char c : fact.toCharArray())
      charSet.add(c); // Autoboxing
    PriorityQueue<Character> characterPQ =
      new PriorityQueue<Character>(charSet);
    QueueDemo.printQ(characterPQ);
  }
} /* Output:
0 1 1 1 1 1 3 5 8 14
1 1 2 3 3 9 9 14 14 18 18 20 21 22 23 25 25
25 25 23 22 21 20 18 18 14 14 9 9 3 3 2 1 1
      A A B C C C D D E E E F H H I I L N N O O O O S S S
T T U U U W
W U U U T T S S S O O O O N N L I I H H F E E E D D C C C B
A A
  A B C D E F H I L N O S T U W
*///:~
```

你可以看到，重复是允许的，最小的值拥有最高的优先级（如果是**String**，空格也可以算作值，并且比字母的优先级高）。为了展示你可以使用怎样的方法通过提供自己的**Comparator**对象来改变排序，第三个对**PriorityQueue<Integer>**的构造器调用，和第二个对**PriorityQueue<String>**的调用使用了由**Collection.reverseOrder()**（新添加到Java SE5中的）产生的反序的**Comparator**。

426

最后一部分添加了一个**HashSet**来消除重复的**Character**，这么做只是为了增添点乐趣。

Integer、**String**和**Character**可以与**PriorityQueue**一起工作，因为这些类已经内建了自然排序。如果你想在**PriorityQueue**中使用自己的类，就必须包括额外的功能以产生自然排序，或者必须提供自己的**Comparator**。在第17章中有一个更加复杂的示例将演示这种情况。

练习28：(2) 用由**java.util.Random**创建的**Double**值填充一个**PriorityQueue**（用**offer()**方法），然后使用**poll()**移除并显示它们。

练习29：(2) 创建一个继承自**Object**的简单类，它不包含任何成员，展示你不能将这个类的多个示例成功地添加到一个**PriorityQueue**中。这个问题将在第17章中详细解释。

11.12 Collection和Iterator

Collection是描述所有序列容器的共性的根接口，它可能会被认为是一个“附属接口”，即因为要表示其他若干个接口的共性而出现的接口。另外，**java.util.AbstractCollection**类提供了**Collection**的默认实现，使得你可以创建**AbstractCollection**的子类型，而其中没有不必要的代码

重复。

使用接口描述的一个理由是它可以使我们能够创建更通用的代码。通过针对接口而非具体实现来编写代码，我们的代码可以应用于更多的对象类型[⊖]。因此，如果我编写的方法将接受一个**Collection**，那么该方法就可以应用于任何实现了**Collection**的类——这也就使得一个新类可以选择去实现**Collection**接口，以便我的方法可以使用它。但是，有一点很有趣，就是我们注意到标准C++类库中并没有其容器的任何公共基类——容器之间的所有共性都是通过迭代器达成的。在Java中，遵循C++的方式看起来似乎很明智，即用迭代器而不是**Collection**来表示容器之间的共性。但是，这两种方法绑定到了一起，因为实现**Collection**就意味着需要提供**iterator()**方法：

427

```
//: holding/InterfaceVsIterator.java
import typeinfo.pets.*;
import java.util.*;

public class InterfaceVsIterator {
  public static void display(Iterator<Pet> it) {
    while(it.hasNext()) {
      Pet p = it.next();
      System.out.print(p.id() + ":" + p + " ");
    }
    System.out.println();
  }
  public static void display(Collection<Pet> pets) {
    for(Pet p : pets)
      System.out.print(p.id() + ":" + p + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    List<Pet> petList = Pets.arrayList(8);
    Set<Pet> petSet = new HashSet<Pet>(petList);
    Map<String,Pet> petMap =
      new LinkedHashMap<String,Pet>();
    String[] names = ("Ralph, Eric, Robin, Lacey, " +
      "Britney, Sam, Spot, Fluffy").split(", ");
    for(int i = 0; i < names.length; i++)
      petMap.put(names[i], petList.get(i));
    display(petList);
    display(petSet);
    display(petList.iterator());
    display(petSet.iterator());
    System.out.println(petMap);
    System.out.println(petMap.keySet());
    display(petMap.values());
    display(petMap.values().iterator());
  }
} /* Output:
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
4:Pug 6:Pug 3:Mutt 1:Manx 5:Cymric 7:Manx 2:Cymric 0:Rat
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
4:Pug 6:Pug 3:Mutt 1:Manx 5:Cymric 7:Manx 2:Cymric 0:Rat
{Ralph=Rat, Eric=Manx, Robin=Cymric, Lacey=Mutt,
Britney=Pug, Sam=Cymric, Spot=Pug, Fluffy=Manx}
[Ralph, Eric, Robin, Lacey, Britney, Sam, Spot, Fluffy]
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
*///:~
```

428

⊖ 某些人提倡这样一种自动创建机制，即对一个类中所有可能的方法组合都自动创建一个接口，有时要针对每一个单个的类都自动创建。我相信接口的意义应该不仅限于方法组合的机械地复制，因此我在创建接口之前，总是要先看到增加接口带来的价值。

两个版本的**display()**方法都可以使用**Map**或**Collection**的子类型来工作，而且**Collection**接口和**Iterator**都可以将**display()**方法与底层容器的特定实现解耦。

在本例中，这两种方式都可以凑效。事实上，**Collection**要更方便一点，因为它是**Iterable**类型，因此，在**display(Collection)**实现中，可以使用foreach结构，从而使代码更加清晰。

当你要实现一个不是**Collection**的外部类时，由于让它去实现**Collection**接口可能非常困难或麻烦，因此使用**Iterator**就会变得非常吸引人。例如，如果我们通过继承一个持有**Pet**对象的类来创建一个**Collection**的实现，那么我们必须实现所有的**Collection**方法，即使我们在**display()**方法中不必使用它们，也必须如此。尽管这可以通过继承**AbstractCollection**而很容易地实现，但是你无论如何还是要被强制去实现**iterator()**和**size()**，以便提供**AbstractCollection**没有实现，但是**AbstractCollection**中的其他方法会使用到的方法：

```
//: holding/CollectionSequence.java
import typeinfo.pets.*;
import java.util.*;

public class CollectionSequence
extends AbstractCollection<Pet> {
  private Pet[] pets = Pets.createArray(8);
  public int size() { return pets.length; }
  public Iterator<Pet> iterator() {
    return new Iterator<Pet>() {
      private int index = 0;
      public boolean hasNext() {
        return index < pets.length;
      }
      public Pet next() { return pets[index++]; }
      public void remove() { // Not implemented
        throw new UnsupportedOperationException();
      }
    };
  }
  public static void main(String[] args) {
    CollectionSequence c = new CollectionSequence();
    InterfaceVsIterator.display(c);
    InterfaceVsIterator.display(c.iterator());
  }
} /* Output:
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
*///:~
```

429

remove()方法是一个“可选操作”，你将在第17章中了解它。这里不必实现它，如果你调用它，它会抛出异常。

从本例中，你可以看到，如果你实现**Collection**，就必须实现**iterator()**，并且只拿实现**iterator()**与继承**AbstractCollection**相比，花费的代价只有略微减少。但是，如果你的类已经继承了其他的类，那么你就不能再继承**AbstractCollection**了。在这种情况下，要实现**Collection**，就必须实现该接口中的所有方法。此时，继承并提供创建迭代器的能力就会显得容易得多了：

```
//: holding/NonCollectionSequence.java
import typeinfo.pets.*;
import java.util.*;

class PetSequence {
  protected Pet[] pets = Pets.createArray(8);
}

public class NonCollectionSequence extends PetSequence {
  public Iterator<Pet> iterator() {
    return new Iterator<Pet>() {
```

430

```
      private int index = 0;
      public boolean hasNext() {
        return index < pets.length;
      }
      public Pet next() { return pets[index++]; }
      public void remove() { // Not implemented
        throw new UnsupportedOperationException();
      }
    };
  }
  public static void main(String[] args) {
    NonCollectionSequence nc = new NonCollectionSequence();
    InterfaceVsIterator.display(nc.iterator());
  }
} /* Output:
0:Rat 1:Manx 2:Cymric 3:Mutt 4:Pug 5:Cymric 6:Pug 7:Manx
*///:~
```

生成**Iterator**是将队列与消费队列的方法连接在一起耦合度最小的方式，并且与实现**Collection**相比，它在序列类上所施加的约束也少得多。

练习30：(5) 修改**CollectionSequence.java**，使其不要继承**AbstractCollection**，而是实现**Collection**。

11.13 Foreach与迭代器

到目前为止，foreach语法主要用于数组，但是它也可以应用于任何**Collection**对象。你实际上已经看到过很多使用**ArrayList**时用到它的示例，下面是一个更通用的证明：

```
//: holding/ForEachCollections.java
// All collections work with foreach.
import java.util.*;

public class ForEachCollections {
  public static void main(String[] args) {
    Collection<String> cs = new LinkedList<String>();
    Collections.addAll(cs,
      "Take the long way home".split(" "));
    for(String s : cs)
      System.out.print("'" + s + "' ");
  }
} /* Output:
'Take' 'the' 'long' 'way' 'home'
*///:~
```

431

由于**cs**是一个**Collection**，所以这段代码展示了能够与foreach一起工作是所有**Collection**对象的特性。

之所以能够工作，是因为Java SE5引入了新的被称为**Iterable**的接口，该接口包含一个能够产生**Iterator**的**iterator()**方法，并且**Iterable**接口被foreach用来在序列中移动。因此如果你创建了任何实现**Iterable**的类，都可以将它用于foreach语句中：

```
//: holding/IterableClass.java
// Anything Iterable works with foreach.
import java.util.*;

public class IterableClass implements Iterable<String> {
  protected String[] words = ("And that is how " +
    "we know the Earth to be banana-shaped.").split(" ");
  public Iterator<String> iterator() {
    return new Iterator<String>() {
      private int index = 0;
      public boolean hasNext() {
```

```
        return index < words.length;
      }
      public String next() { return words[index++]; }
      public void remove() { // Not implemented
        throw new UnsupportedOperationException();
      }
    };
  }
  public static void main(String[] args) {
    for(String s : new IterableClass())
      System.out.print(s + " ");
  }
} /* Output:
And that is how we know the Earth to be banana-shaped.
*///:~
```

iterator()方法返回的是实现了**Iterator<String>**的匿名内部类的实例，该匿名内部类可以遍历数组中的所有单词。在**main()**中，你可以看到**IterableClass**确实可以用于foreach语句中。

在Java SE5中，大量的类都是**Iterable**类型，主要包括所有的**Collection**类（但是不包括各种**Map**）。例如，下面的代码可以显示所有的操作系统环境变量：

432

```
//: holding/EnvironmentVariables.java
import java.util.*;

public class EnvironmentVariables {
  public static void main(String[] args) {
    for(Map.Entry entry: System.getenv().entrySet()) {
      System.out.println(entry.getKey() + ": " +
        entry.getValue());
    }
  }
} /* (Execute to see output) *///:~
```

System.getenv()[⊖]返回一个**Map**，**entrySet()**产生一个由**Map.Entry**的元素构成的**Set**，并且这个**Set**是一个**Iterable**，因此它可以用于foreach循环。

foreach语句可以用于数组或其他任何**Iterable**，但是这并不意味着数组肯定也是一个**Iterable**，而任何自动包装也不会自动发生：

```
//: holding/ArrayIsNotIterable.java
import java.util.*;

public class ArrayIsNotIterable {
  static <T> void test(Iterable<T> ib) {
    for(T t : ib)
      System.out.print(t + " ");
  }
  public static void main(String[] args) {
    test(Arrays.asList(1, 2, 3));
    String[] strings = { "A", "B", "C" };
    // An array works in foreach, but it's not Iterable:
    //! test(strings);
    // You must explicitly convert it to an Iterable:
    test(Arrays.asList(strings));
  }
} /* Output:
1 2 3 A B C
*///:~
```

433

尝试把数组当作一个**Iterable**参数传递会导致失败。这说明不存在任何从数组到**Iterable**的

⊖ 在Java SE5之前还没有它，因为该方法被认为与操作系统的耦合度过紧，因此会违反“编写一次，到处运行”的原则。现在提供它这一事实表明，Java的设计者们更加务实了。

自动转换，你必须手工执行这种转换。

练习31：(3) 修改**polymorphism/shape/RandomShapeGenerator.java**，使其成为一个**Iterable**。你需要添加一个接收元素数量为参数的构造器，这个数量是指在停止之前，你想用迭代器生成的元素的数量。验证这个程序可以工作。

11.13.1 适配器方法惯用法

如果现有一个**Iterable**类，你想要添加一种或多种在foreach语句中使用这个类的方法，应该怎么做呢？例如，假设你希望可以选择以向前的方向或是向后的方向迭代一个单词列表。如果直接继承这个类，并覆盖**iterator()**方法，你只能替换现有的方法，而不能实现选择。

一种解决方案是所谓*适配器方法*的惯用法。“适配器”部分来自于设计模式，因为你必须提供特定接口以满足foreach语句。当你有一个接口并需要另一个接口时，编写适配器就可以解决问题。这里，我希望在默认的前向迭代器的基础上，添加产生反向迭代器的能力，因此我不能使用覆盖，而是添加了一个能够产生**Iterable**对象的方法，该对象可以用于foreach语句。正如你所见，这使得我们可以提供多种使用foreach的方式：

```
//: holding/AdapterMethodIdiom.java
// The "Adapter Method" idiom allows you to use foreach
// with additional kinds of Iterables.
import java.util.*;

class ReversibleArrayList<T> extends ArrayList<T> {
  public ReversibleArrayList(Collection<T> c) { super(c); }
  public Iterable<T> reversed() {
    return new Iterable<T>() {
      public Iterator<T> iterator() {
        return new Iterator<T>() {
          int current = size() - 1;
          public boolean hasNext() { return current > -1; }
          public T next() { return get(current--); }
          public void remove() { // Not implemented
            throw new UnsupportedOperationException();
          }
        };
      }
    };
  }
}

public class AdapterMethodIdiom {
  public static void main(String[] args) {
    ReversibleArrayList<String> ral =
      new ReversibleArrayList<String>(
        Arrays.asList("To be or not to be".split(" ")));
    // Grabs the ordinary iterator via iterator():
    for(String s : ral)
      System.out.print(s + " ");
    System.out.println();
    // Hand it the Iterable of your choice
    for(String s : ral.reversed())
      System.out.print(s + " ");
  }
} /* Output:
To be or not to be
be to not or be To
*///:~
```

434

如果直接将**ral**对象置于foreach语句中，将得到（默认的）前向迭代器。但是如果在该对象上调用**reversed()**方法，就会产生不同的行为。

通过使用这种方式，我可以在**IterableClass.java**示例中添加两种适配器方法：

```
//: holding/MultiIterableClass.java
// Adding several Adapter Methods.
import java.util.*;

public class MultiIterableClass extends IterableClass {
  public Iterable<String> reversed() {
    return new Iterable<String>() {
      public Iterator<String> iterator() {
        return new Iterator<String>() {
          int current = words.length - 1;
          public boolean hasNext() { return current > -1; }
          public String next() { return words[current--]; }
          public void remove() { // Not implemented
            throw new UnsupportedOperationException();
          }
        };
      }
    };
  }
  public Iterable<String> randomized() {
    return new Iterable<String>() {
      public Iterator<String> iterator() {
        List<String> shuffled =
          new ArrayList<String>(Arrays.asList(words));
        Collections.shuffle(shuffled, new Random(47));
        return shuffled.iterator();
      }
    };
  }
  public static void main(String[] args) {
    MultiIterableClass mic = new MultiIterableClass();
    for(String s : mic.reversed())
      System.out.print(s + " ");
    System.out.println();
    for(String s : mic.randomized())
      System.out.print(s + " ");
    System.out.println();
    for(String s : mic)
      System.out.print(s + " ");
  }
} /* Output:
banana-shaped. be to Earth the know we how is that And
is banana-shaped. Earth that how the be And we know to
And that is how we know the Earth to be banana-shaped.
*///:~
```

435

注意，第二个方法**random()**没有创建它自己的**Iterator**，而是直接返回被打乱的**List**中的**Iterator**。

从输出中可以看到，**Collection.shuffle()**方法没有影响到原来的数组，而只是打乱了**shuffled**中的引用。之所以这样，只是因为**randomized()**方法用一个**ArrayList**将**Arrays.asList()**方法的结果包装了起来。如果这个由**Arrays.asList()**方法产生的**List**被直接打乱，那么它就会修改底层的数组，就像下面这样：

436

```
//: holding/ModifyingArraysAsList.java
import java.util.*;
public class ModifyingArraysAsList {
  public static void main(String[] args) {
    Random rand = new Random(47);
    Integer[] ia = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    List<Integer> list1 =
      new ArrayList<Integer>(Arrays.asList(ia));
    System.out.println("Before shuffling: " + list1);
    Collections.shuffle(list1, rand);
    System.out.println("After shuffling: " + list1);
```

```
    System.out.println("array: " + Arrays.toString(ia));

    List<Integer> list2 = Arrays.asList(ia);
    System.out.println("Before shuffling: " + list2);
    Collections.shuffle(list2, rand);
    System.out.println("After shuffling: " + list2);
    System.out.println("array: " + Arrays.toString(ia));
  }
} /* Output:
Before shuffling: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
After shuffling: [4, 6, 3, 1, 8, 7, 2, 5, 10, 9]
array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Before shuffling: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
After shuffling: [9, 1, 6, 3, 7, 2, 5, 10, 4, 8]
array: [9, 1, 6, 3, 7, 2, 5, 10, 4, 8]
*///:~
```

在第一种情况中，**Arrays.asList()**的输出被传递给了**ArrayList()**的构造器，这将创建一个引用**ia**的元素的**ArrayList**，因此打乱这些引用不会修改该数组。但是，如果直接使用**Arrays.asList(ia)**的结果，这种打乱就会修改**ia**的顺序。意识到**Arrays.asList()**产生的**List**对象会使用底层数组作为其物理实现是很重要的。只要你执行的操作会修改这个**List**，并且你不想原来的数组被修改，那么你就应该在另一个容器中创建一个副本。

练习32：(2) 按照**MultiIterableClass**示例，在**NonCollectionSequence.java**中添加**reversed()**和**randomized()**方法，并让**NonCollectionSequence**实现**Iterable**。然后在foreach语句中展示所有的使用方式。

11.14 总结

Java提供了大量持有对象的方式：

1) 数组将数字与对象联系起来。它保存类型明确的对象，查询对象时，不需要对结果做类型转换。它可以是多维的，可以保存基本类型的数据。但是，数组一旦生成，其容量就不能改变。

2) **Collection**保存单一的元素，而**Map**保存相关联的键值对。有了Java的泛型，你就可以指定容器中存放的对象类型，因此你就不会将错误类型的对象放置到容器中，并且在从容器中获取元素时，不必进行类型转换。各种**Collection**和各种**Map**都可以在你向其中添加更多的元素时，自动调整其尺寸。容器不能持有基本类型，但是自动包装机制会仔细地执行基本类型到容器中所持有的包装器类型之间的双向转换。

3) 像数组一样，**List**也建立数字索引与对象的关联，因此，数组和**List**都是排好序的容器。List能够自动扩充容量。

4) 如果要进行大量的随机访问，就使用**ArrayList**；如果要经常从表中间插入或删除元素，则应该使用**LinkedList**。

5) 各种**Queue**以及栈的行为，由**LinkedList**提供支持。

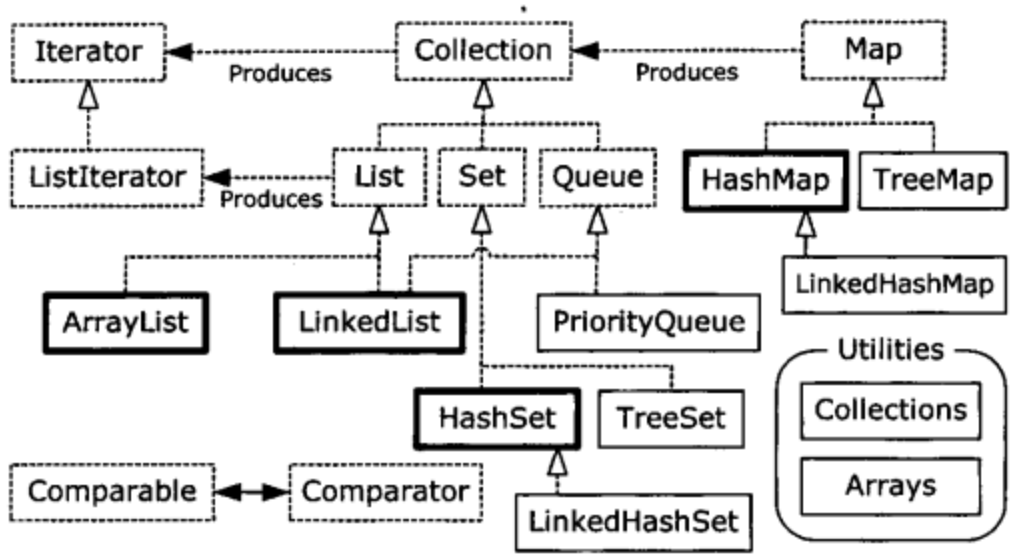
6) **Map**是一种将对象（而非数字）与对象相关联的设计。**HashMap**设计用来快速访问；而**TreeMap**保持“键”始终处于排序状态，所以没有**HashMap**快。**LinkedHashMap**保持元素插入的顺序，但是也通过散列提供了快速访问能力。

7) **Set**不接受重复元素。**HashSet**提供最快的查询速度，而**TreeSet**保持元素处于排序状态。**LinkedHashSet**以插入顺序保存元素。

8) 新程序中不应该使用过时的**Vector**、**Hashtable**和**Stack**。

浏览一下Java容器的简图（不包含抽象类和遗留构件）会大有裨益。这里只包含你在一般

438 情况下会碰到的接口和类。



简单的容器分类

你可以看到，其实只有四种容器：**Map**、**List**、**Set**和**Queue**，它们各有两到三个实现版本（**Queue**的**java.util.concurrent**实现没有包括在上面这张图中）。常用的容器用黑色粗线框表示。

点线框表示接口，实线框表示普通的（具体的）类。带有空心箭头的点线表示一个特定的类实现了一个接口，实心箭头表示某个类可以生成箭头所指向类的对象。例如，任意的**Collection**可以生成**Iterator**，而**List**可以生成**ListIterator**（也能生成普通的**Iterator**，因为List继承自**Collection**）。

下面的示例展示了各种不同的类在方法上的差异。实际的代码来自第15章，我在这里只是调用它以产生输出。程序的输出也展示了在每个类或接口中所实现的接口：

```
//: holding/ContainerMethods.java
import net.mindview.util.*;

public class ContainerMethods {
  public static void main(String[] args) {
    ContainerMethodDifferences.main(args);
  }
} /* Output: (Sample)
Collection: [add, addAll, clear, contains, containsAll,
equals, hashCode, isEmpty, iterator, remove, removeAll,
retainAll, size, toArray]
Interfaces in Collection: [Iterable]
Set extends Collection, adds: []
Interfaces in Set: [Collection]
HashSet extends Set, adds: []
Interfaces in HashSet: [Set, Cloneable, Serializable]
LinkedHashSet extends HashSet, adds: []
Interfaces in LinkedHashSet: [Set, Cloneable, Serializable]
TreeSet extends Set, adds: [pollLast, navigableHeadSet,
descendingIterator, lower, headSet, ceiling, pollFirst,
subSet, navigableTailSet, comparator, first, floor, last,
navigableSubSet, higher, tailSet]
Interfaces in TreeSet: [NavigableSet, Cloneable,
Serializable]
List extends Collection, adds: [listIterator, indexOf, get,
subList, set, lastIndexOf]
Interfaces in List: [Collection]
ArrayList extends List, adds: [ensureCapacity, trimToSize]
Interfaces in ArrayList: [List, RandomAccess, Cloneable,
Serializable]
LinkedList extends List, adds: [pollLast, offer,
descendingIterator, addFirst, peekLast, removeFirst,
```

439

```
peekFirst, removeLast, getLast, pollFirst, pop, poll,
addLast, removeFirstOccurrence, getFirst, element, peek,
offerLast, push, offerFirst, removeLastOccurrence]
Interfaces in LinkedList: [List, Deque, Cloneable,
Serializable]
Queue extends Collection, adds: [offer, element, peek,
poll]
Interfaces in Queue: [Collection]
PriorityQueue extends Queue, adds: [comparator]
Interfaces in PriorityQueue: [Serializable]
Map: [clear, containsKey, containsValue, entrySet, equals,
get, hashCode, isEmpty, keySet, put, putAll, remove, size,
values]
HashMap extends Map, adds: []
Interfaces in HashMap: [Map, Cloneable, Serializable]
LinkedHashMap extends HashMap, adds: []
Interfaces in LinkedHashMap: [Map]
SortedMap extends Map, adds: [subMap, comparator, firstKey,
lastKey, headMap, tailMap]
Interfaces in SortedMap: [Map]
TreeMap extends Map, adds: [descendingEntrySet, subMap,
pollLastEntry, lastKey, floorEntry, lastEntry, lowerKey,
navigableHeadMap, navigableTailMap, descendingKeySet,
tailMap, ceilingEntry, higherKey, pollFirstEntry,
comparator, firstKey, floorKey, higherEntry, firstEntry,
navigableSubMap, headMap, lowerEntry, ceilingKey]
Interfaces in TreeMap: [NavigableMap, Cloneable,
Serializable]
*///:~
```

440

可以看到，除了**TreeSet**之外的所有**Set**都拥有与**Collection**完全一样的接口。**List**和**Collection**存在着明显的不同，尽管**List**所要求的方法都在**Collection**中。另一方面，在**Queue**接口中的方法都是独立的；在创建具有**Queue**功能的实现时，不需要使用**Collection**方法。最后，**Map**和**Collection**之间的唯一重叠就是**Map**可以使用**entrySet()**和**values()**方法来产生**Collection**。

注意，标记接口**java.util.RandomAccess**附着到了**ArrayList**上，而没有附着到**LinkedList**上。这为想要根据所使用的特定的**List**而动态修改其行为的算法提供了信息。

从面向对象的继承层次关系来看，这种组织结构确实有些奇怪。但是，当你了解了**java.util**中更多的有关容器的内容后（特别是第17章中的内容），你就会看到除了继承结构有些奇怪外，还有更多的问题。容器类库一直以来都是设计难题——解决这些难题涉及到要去满足经常彼此之间互为牵制的各方面需求。因此你应该学会中庸之道。

抛开这些问题，Java的容器每天都会用到的工具，它可以使程序更简洁、更强大、更高效。在适应容器类库的某些方面之前，你确实得废点劲儿，但是我想你很快就会找到自己的路子，去获得和使用这个类库中的类。

441 ~ 442

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。